Node.js CRUD Application - Complete Project Documentation

Table of Contents

Project Overview

- This is a production-ready **Node.js CRUD REST API** application with a responsive frontend dashboard for managing products. The application is fully containerized with Docker and deployed on **AWS EC2** with automated CI/CD using **GitHub Actions** and a self-hosted runner.

Key Features

- Complete CRUD operations for product management
- RESTful API with Express.js
- MongoDB Atlas cloud database integration
- Docker containerization with health checks
- Nginx reverse proxy
- Automated CI/CD pipeline with self-hosted runner
- Production-ready error handling and validation
- Comprehensive testing with Jest
- ESLint code quality checks
- Responsive web interface

Technologies Used

Backend Technologies

Technology	Version	Purpose
Node.js	v22.17.1	JavaScript runtime environment
Express.js	express@4.22.1	Web Application framework
MongoDB	Atlas Cloud	NoSQL database
Mongoose	mongoose@8.22.0	MongoDB ODM
CORS	cors@2.8.6	Cross-Origin Resource Sharing
.env	dotenv@16.6.1	Environment variable management

DevOps & Infrastructure

Technology	Version	Purpose
Docker	28.3.2	Containerization platform
Docker Compose	v2.38.2-desktop.1	Multi-container orchestration

Nginx	Latest	Reverse proxy & web server
AWS EC2	t2.micro	Cloud hosting (Ubuntu 22.04 LTS)
GitHub Actions	N/A	CI/CD Automation
Self-hosted Runner	v2.331.0	GitHub Actions runner on EC2

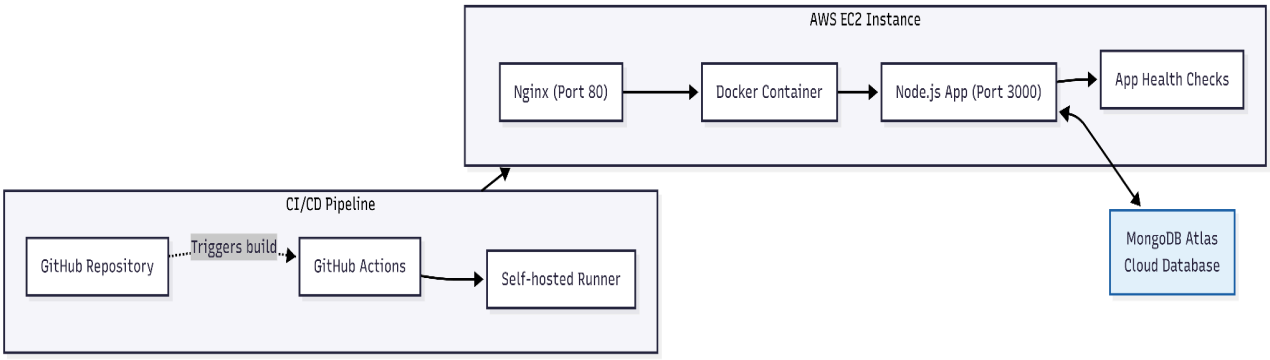
Development Tools

Technology	**Version**	**Purpose**
Jest	jest@29.7.0	
Supertest	supertest@6.	Testing framework
ESLint	eslint@8.57.1	Code linting & quality
Nodemon	nodemon@3.1.11	Development auto-reload

Frontend

Technology	Purpose
HTML5	Structure
CSS3	Styling with modern design
Vanilla JavaScript	Frontend logic (no frameworks)
Fetch API	Asynchronous API calls

🏗️ Architecture



Application Flow

User accesses application via browser → `http://3.110.113.159/`

Nginx receives request on port 80 → forwards to port 3000

Express.js handles API requests → interacts with MongoDB

MongoDB Atlas stores and retrieves data

Response flows back through the same chain

File Structure

nodejs-crud-clean/

```
├── src/
│   ├── config/
│   │   └── db.js # MongoDB connection logic
│   ├── controllers/
│   │   └── product.controller.js # Business logic
│   ├── models/
│   │   └── product.model.js # Mongoose schema
│   ├── routes/
│   │   └── product.routes.js # API route definitions
│   ├── app.js # Express app configuration
│   └── server.js # Application entry point
├── tests/
│   └── product.test.js # Jest test suite
├── public/
│   └── index.html # Frontend dashboard
├── .github/
│   └── workflows/
│   └── deploy.yml # CI/CD pipeline
├── docker-compose.yml # Container orchestration
├── Dockerfile # Container build instructions
├── package.json # Dependencies & scripts
├── jest.config.js # Test configuration
├── .env # Environment variables
├── README.md # Project overview
├── DEPLOYMENT.md # Deployment guide
```

└── **PROJECT_DOCUMENTATION.md** # This file

🚀 Development Setup

Prerequisites

- Node.js (v22)
- MongoDB Atlas account
- Git
- Code editor (VS Code recommended)

Step 1: Clone Repository

```bash

- ❖ git clone https://github.com/Kaveera725/nodejs-crud-clean.git
- ❖ cd nodejs-crud-clean

### Step 2: Install Dependencies

```bash

- ❖ npm install

### Step 3: Environment Configuration

- ❖ Create `.env` file in project root:

.env

```
PORT=3000

NODE_ENV=development

MONGO_URI=mongodb+srv://crud_user:crud_user@cluster0.e0402j9.mongodb
.net/crud_db?retryWrites=true&w=majority&appName=Cluster0
```

Step 4: MongoDB Atlas Setup

1. Create account at
[mongodb.com/cloud/atlas](https://www.mongodb.com/cloud/atlas)

2. Create a cluster (Free M0 tier available)

3. Configure database access (create user)

4. Configure network access:

- Development: Add your current IP

- Production: Add `0.0.0.0/0` (EC2 access)

5. Get connection string and update `.env`

Step 5: Run Development Server

```bash

# Start with auto-reload

- npm run dev

# Or start normally

- npm start

Application runs at: `http://localhost:3000`

### Step 6: Run Tests

```bash

# Run all tests with coverage

- npm test

# Run linting

- npm run lint

# Auto-fix linting issues

- npm run lnt:fixi

---

## 🌐 Production Deployment

### Step 1: Launch AWS EC2 Instance

**Instance Configuration:**

- Name: nodejs-crud-app
- AMI: Ubuntu Server 22.04 LTS
- Instance Type: t2.micro (Free tier)
- Key Pair: Create/download .pem file
- Security Groups:
- SSH (22) - Your IP only
- HTTP (80) - 0.0.0.0/0 (anywhere)
- HTTPS (443) - 0.0.0.0/0 (optional)

**Storage:8 GB**

**Public IP Address: `3.110.113.159`**

**### Step 2: Connect to EC2**

```bash

# Windows PowerShell/Git Bash

- ssh -i "your-key.pem" ubuntu@3.110.113.159
```

Step 3: Install Docker

```bash

# Update system packages

- sudo apt update && sudo apt upgrade -y

# Install Docker

- sudo apt install docker.io -y

# Start Docker service

- sudo systemctl start docker
- sudo systemctl enable docker

# Add user to docker group

- sudo usermod -aG docker ubuntu

# Verify installation
```

- docker --version

Output: Docker version 28.2.2

Install Docker Compose

- sudo curl -L "https://github.com/docker/compose/releases/latest/download/docker-compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
- sudo chmod +x /usr/local/bin/docker-compose
- docker-compose --version

Step 4: Clone Repository on EC2

```bash

- git clone https://github.com/Kaveera725/nodejs-crud-clean.git
- cd nodejs-crud-clean

```

Step 5: Configure Environment Variables

```bash

- nano .env
- Add production configuration:

Env

PORT=3000

NODE_ENV=production

MONGO_URI=mongodb+srv://crud_user:crud_user@cluster0.e0402j9.mongodb.net/crud_db?retryWrites=true&w=majority&appName=Cluster0
```

Step 6: Deploy with Docker

```bash

# Build and start containers
```

- docker-compose up -d

Check container status

- docker ps

View logs

- docker-compose logs -f

Expected Output:

- nodejs-crud-app running healthy 0.0.0.0:80->3000/tcp

```

**### Step 7: Verify Deployment**

```bash

# Test health endpoint

- curl http://localhost/health

# Expected: {"status":"ok","message":"Server is running"}

# Check MongoDB connection in logs

- docker-compose logs | grep "MongoDB"

# Expected: "MongoDB Atlas connected successfully"

Access Application:

- Open browser: `http://3.110.113.159/`
- Server status should show: **Online** ✅
- Test CRUD operations through the dashboard

---

## ⚙️ CI/CD Pipeline

### Step 1: Setup Self-hosted GitHub Actions Runner

**Why Self-hosted Runner?**

- Direct access to EC2 instance (no SSH required)

- Faster deployments
- No external SSH key management
- Free (no GitHub Actions minutes consumed)

**Install Runner on EC2:**

```bash

# Create runner directory

- mkdir actions-runner && cd actions-runner

# Download runner (replace with latest version)

- curl -o actions-runner-linux-x64-2.331.0.tar.gz -L \
- https://github.com/actions/runner/releases/download/v2.331.0/actions-runner-linux-x64-2.331.0.tar.gz

# Extract

- tar xzf ./actions-runner-linux-x64-2.331.0.tar.gz

# Configure runner

- ./config.sh --url https://github.com/Kaveera725/nodejs-crud-clean--token YOUR_REGISTRATION_TOKEN

# Install as system service

- sudo ./svc.sh install
- sudo ./svc.sh start

# Verify service status

- sudo systemctl status actions.runner.

```

Get Registration Token:

1. Go to GitHub repository
2. Settings → Actions → Runners

3. Click "New self-hosted runner"

4. Copy registration token (valid for 1 hour)

Step 2: Create GitHub Secrets

Repository Settings → Secrets and variables → Actions → New secret

Add secret:

- Name: `MONGO_URI`
- Value: Your MongoDB Atlas connection string

Step 3: Create Workflow File

File: `.github/workflows/deploy.yml`

```yaml
name: Deploy to EC2

on:

push:

branches: [ main ]

workflow_dispatch:

jobs:

deploy:

runs-on: self-hosted

steps:

- name: Checkout code

uses: actions/checkout@v3

- name: Create .env file

run: |

echo "PORT=3000" > .env

echo "NODE_ENV=production" >> .env
```

```
      echo "MONGO_URI=${{ secrets.MONGO_URI }}" >> .env

      - name: Deploy with Docker Compose

      run: |

      docker-compose down

      docker-compose up -d --build

      - name: Check deployment

      run: |

      sleep 10

      curl -f http://localhost/health || exit 1

      - name: Show container status

      run: docker ps
```

Step 4: Deploy Application

```bash

# Commit and push workflow file
```

- git add .github/workflows/deploy.yml
- git commit -m "Add CI/CD pipeline"
- git push origin main

Monitoring Deployments:

- GitHub Repository → Actions tab
- View workflow runs and logs
- Runner status: Idle/Active

Automated Deployment Process

1. Developer pushes code to `main` branch
2. GitHub triggers workflow on self-hosted runner
3. Runner pulls latest code on EC2

4. Creates `.env` file with GitHub secrets

5. Stops existing containers

6. Rebuilds and starts new containers

7. Performs health check

8. Deployment complete! ✅

🔧 Troubleshooting

Common Issues & Solutions

1. "Server Status: Offline" on Frontend

Problem: Frontend showing offline despite backend running

Diagnosis:

```bash

# Check container status

- docker ps

# Check logs

- docker-compose logs
- Solution: Fixed hardcoded localhost URLs in `public/index.html`

javascript

// ❌ WRONG (caused CORS errors)

- const API_URL = 'http://localhost:3000/api/products';
- const HEALTH_URL = 'http://localhost:3000/health';

// ✅ CORRECT (relative paths)

- const API_URL = '/api/products';
- const HEALTH_URL = '/health';

**2. MongoDB Connection Failed**
```

Problem: "MongoServerError: Authentication failed"

Solution:

bash

Verify MONGO_URI in .env

- cat .env | grep MONGO_URI

Check MongoDB Atlas network access

Add 0.0.0.0/0 in Atlas dashboard

Restart containers

- docker-compose restart

3. GitHub Actions Runner 404 Error

Problem: "A 404 error occurred while accessing the runner registration endpoint"

Cause: Registration token expired (1-hour lifetime)

Solution:

1. Generate new token: GitHub → Settings → Actions → Runners
2. Re-run configuration:

```bash

- cd ~/actions-runner
- ./config.sh --url https://github.com/YOUR_USERNAME/YOUR_REPO --token NEW_TOKEN

**4. Port Already in Use**

Problem: "Error: Port 3000 is already in use"

Solution:

```bash

# Find process using port

- sudo lsof -i :3000

# Kill the process

- sudo kill -9 PID

# Or use Docker to clean up

- docker-compose down
- docker-compose up -d

**5. Container Health Check Failing**

Problem: Container showing "unhealthy" status

Diagnosis:

```bash

# Check health endpoint manually

- docker exec nodejs-crud-app curl http://localhost:3000/health

# View detailed logs

- docker-compose logs --tail=50

```

Solution:

- Ensure MongoDB connection is successful
- Verify `.env` file has correct `MONGO_URI`
- Check that port 3000 is accessible inside container

📚 API Documentation

Base URL

- Development: `http://localhost:3000`
- Production: `http://3.110.113.159`

Endpoints

Health Check

```http
GET /health
```

Response:

```json
{
"status": "ok",
"message": "Server is running"
}
```

Get All Products

```http
GET /api/products
```

Response:

```json
[
{
"_id": "65f1a2b3c4d5e6f7g8h9i0j1",
"name": "Laptop",
"description": "High-performance laptop",
"price": 999.99,
"category": "Electronics",
"createdAt": "2026-02-05T10:30:00.000Z",
```

"updatedAt": "2026-02-05T10:30:00.000Z"

}

]

```

---

#### Get Single Product

```http

GET /api/products/:id

```

Response:

```json

{

"_id": "65f1a2b3c4d5e6f7g8h9i0j1",

"name": "Laptop",

"description": "High-performance laptop",

"price": 999.99,

"category": "Electronics",

"createdAt": "2026-02-05T10:30:00.000Z",

"updatedAt": "2026-02-05T10:30:00.000Z"

}

**#### Create Product**

**```http**

POST /api/products

Content-Type: application/json

```
{
"name": "Laptop",
"description": "High-performance laptop",
"price": 999.99,
"category": "Electronics"
}
```

Response: `201 Created`

```json
{
"_id": "65f1a2b3c4d5e6f7g8h9i0j1",
"name": "Laptop",
"description": "High-performance laptop",
"price": 999.99,
"category": "Electronics",
"createdAt": "2026-02-05T10:30:00.000Z",
"updatedAt": "2026-02-05T10:30:00.000Z"
}
```

Update Product

```http
PUT /api/products/:id
Content-Type: application/json
{
```

"name": "Updated Laptop",

"price": 899.99

}

```

**Response:** `200 OK`

```json

{

"_id": "65f1a2b3c4d5e6f7g8h9i0j1",

"name": "Updated Laptop",

"description": "High-performance laptop",

"price": 899.99,

"category": "Electronics",

"updatedAt": "2026-02-05T11:15:00.000Z"

}

#### Delete Product

```http

DELETE /api/products/:id

```

Response: `200 OK`

```json

{

"message": "Product deleted successfully"

}
```

Error Responses

400 Bad Request:

```json
{
"message": "Validation error",
"errors": ["Name is required", "Price must be a positive number"]
}
```

404 Not Found:

```json
{
"message": "Product not found"
}
```

500 Internal Server Error:

```json
{
"message": "Internal server error",
"error": "Error details..."
}
```

🎓 Lessons Learned

1. Self-hosted Runners: Eliminate SSH complexity and provide direct EC2 access
2. Relative URLs: Always use relative paths in frontend for production compatibility

3. Health Checks: Essential for container orchestration and deployment verification
4. Environment Variables: Never hardcode credentials; use secrets management
5. CORS Configuration: Plan for cross-origin scenarios in production
6. MongoDB Atlas: Whitelist `0.0.0.0/0` for dynamic cloud instance IPs
7. Docker Networking: Nginx reverse proxy enables clean port mapping (80→3000)

🚀 **Future Enhancements**

- ❖ Add SSL/TLS certificate (HTTPS)
- ❖ Implement custom domain name
- ❖ Add Redis caching layer
- ❖ Implement rate limiting
- ❖ Add user authentication (JWT)
- ❖ Create admin dashboard
- ❖ Add product image uploads
- ❖ Implement search and pagination
- ❖ Add monitoring with Prometheus/Grafana
- ❖ Setup automated backups
- ❖ Add load balancer for scaling

📝 **License**

MIT License - See LICENSE file for details

👤 **Author**

Project: Node.js CRUD Clean Architecture

- Repository:

  github.com/Kaveera725/nodejs-crud-clean
- Deployment: AWS EC2 with Docker & GitHub Actions

- Live URL: http://3.110.113.159/

Last Updated: February 5, 2026